

# LABORATORY REPORT

## Laboratory Activity No. 1: Real-Time Multisensor Room Monitoring System

**Course:** BCA152 Microcontrollers

**Institution:** Department of Computer Applications, College of Computer Studies, Mindanao State University - Iligan Institute of Technology

**Instructor:** Asst. Prof. Paul Rodolf P. Castor, M.Sc.

**Date:** September 2026

### 1. Problem and Requirements

Modern embedded applications deployed in smart environmental monitoring environments require real-time execution of diverse physical sensors, interactive user display controls, safety alarm evaluations, and power state management. In this activity, a simulated FreeRTOS-Based ESP32 Room Multisensor System is designed and implemented using PlatformIO with the native Espressif IoT Development Framework (ESP-IDF) and simulated within the Wokwi simulation environment.

#### *1.1 Functional Requirements*

- **FR-01 (Temperature Measurement):** Periodically acquire temperature readings via DHT22 sensor.
- **FR-02 (Humidity Measurement):** Periodically acquire relative humidity readings via DHT22 sensor.
- **FR-03 (Ambient-Light Measurement):** Monitor relative ambient light level using a Photoresistor (LDR) connected to an ESP32 ADC channel (scaled 0-100%).
- **FR-04 (Motion Detection):** Detect physical room movement using a PIR motion sensor.
- **FR-05 (OLED Display):** Render the currently selected sensor value on an SSD1306 OLED screen over I2C.
- **FR-06 (Rotary Encoder Navigation):** Support page traversal (Temperature -> Humidity -> Light -> Motion) using clockwise and counterclockwise rotary encoder signals.

- **FR-07 (Temperature Alarm):** Trigger an audible buzzer alarm when temperature drops below 18 °C (LOW\_TEMP) or exceeds 30 °C (HIGH\_TEMP).
- **FR-08 (Activity State Management):** Support distinct ACTIVE and INACTIVE power states.
- **FR-09 (Automatic Inactivity):** Automatically enter INACTIVE state if no motion is detected for 15 seconds.
- **FR-10 (Automatic Reactivation):** Restore system state to ACTIVE upon PIR motion detection.

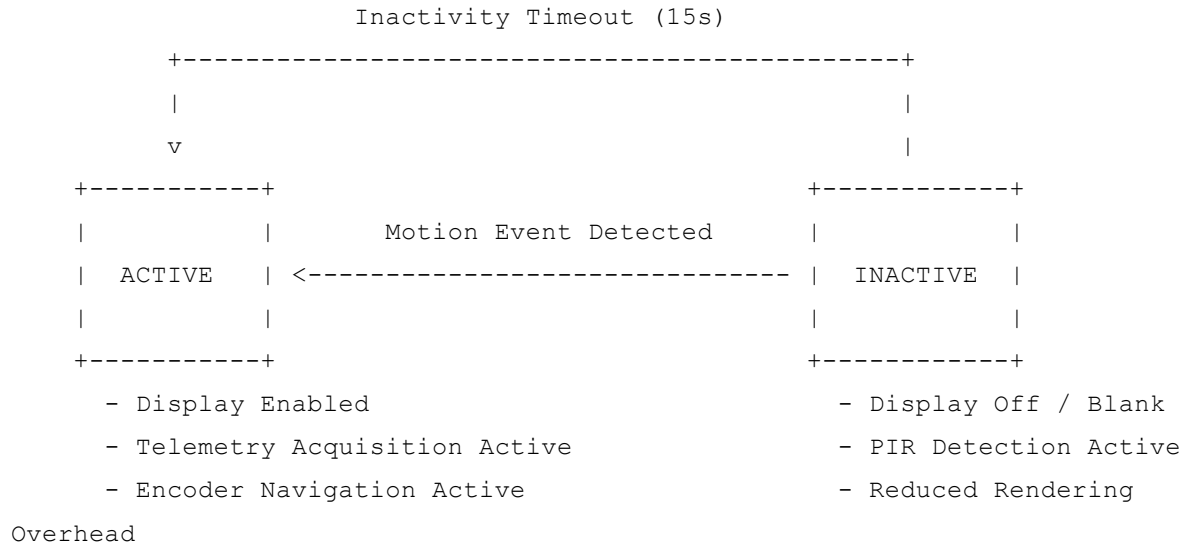
## 2. System Architecture and Design

The software architecture isolates hardware driver routines, synchronization primitives, and deterministic decision logic into modular components. Hardware access is distributed across specialized FreeRTOS tasks to prevent blocking bugs and race conditions.

### 2.1 Software Subsystem Decomposition

| Subsystem      | Header / Source Files                  | Primary Responsibility                                                 |
|----------------|----------------------------------------|------------------------------------------------------------------------|
| Sensors        | sensors.h / .cpp                       | Executes DHT22 reading and ADC conversion for LDR.                     |
| Display        | display.h / .cpp                       | Manages SSD1306 OLED layout formatting and rendering.                  |
| Input          | input.h / .cpp                         | Processes rotary encoder state changes and page indexes.               |
| Alarm          | alarm.h / .cpp                         | Evaluates temperature limits and toggles buzzer GPIO pin.              |
| Motion & State | motion.h / .cpp, system_state.h / .cpp | Monitors PIR signal transitions and manages ACTIVE/INACTIVE state.     |
| RTOS Objects   | rtos_objects.h / .cpp                  | Instantiates FreeRTOS queues, mutexes, event groups, and task handles. |

## 2.2 System State Machine



## 3. FreeRTOS Architecture

### 3.1 Required FreeRTOS Task Table

| Task Name   | Primary Responsibility         | Trigger / Period      | Priority | IPC Primitives            | Blocked Condition              |
|-------------|--------------------------------|-----------------------|----------|---------------------------|--------------------------------|
| MotionTask  | PIR activity monitoring        | Periodic / Edge       | 3        | Event Group, Notification | Waiting for PIR pulse or delay |
| InputTask   | Rotary encoder navigation      | Interrupt / 50 ms     | 3        | Event Group, Queue        | Waiting for GPIO edge or delay |
| SensorTask  | DHT22 & LDR acquisition        | Periodic (2000 ms)    | 2        | Queue, Mutex              | Blocked on vTaskDelayUntil()   |
| AlarmTask   | Threshold evaluation & buzzer  | Queue Event           | 2        | Queue                     | Waiting on xQueueReceive()     |
| DisplayTask | Sole ownership of SSD1306 OLED | Queue / Refresh Event | 1        | Queue, Event Group        | Waiting on xQueueReceive()     |

### ***3.2 Technical Task Priority Justification***

- **Priority 3 (High — MotionTask, InputTask):** Immediate response is required for rotary encoder turns and PIR motion detection. Assigning priority 3 prevents UI navigation latency and ensures the device wakes instantly upon motion detection.
- **Priority 2 (Medium — SensorTask, AlarmTask):** Environmental variables change slowly (sampled every 2 seconds). Alarm processing is critical upon data arrival, but can tolerate minor millisecond scheduling delays compared to physical user inputs.
- **Priority 1 (Low — DisplayTask):** Rendering pages via I2C is computationally slow. Running DisplayTask at priority 1 ensures display refreshes never starve time-critical telemetry acquisition or interrupt processing.

### ***3.3 Synchronization Primitives***

- **Queue (xSensorQueue):** Safely transmits SensorData structs from SensorTask to consumer tasks (DisplayTask, AlarmTask), avoiding global data corruption.
- **Mutex (xSerialMutex):** Grants exclusive ownership of the UART serial port, preventing garbled output when multiple tasks log concurrently.
- **Event Group (xSystemEventGroup):** Manages global system flags: BIT0 (EVENT\_ACTIVE) set when in ACTIVE state; BIT1 (EVENT\_MOTION) pulsed on motion; BIT2 (EVENT\_ALARM) asserted on temperature alarm.

## **4. Implementation Decisions**

### ***4.1 Timing Analysis: vTaskDelayUntil() vs. vTaskDelay()***

Using `vTaskDelay(2000ms)` causes phase drift over time because execution time spent reading sensor hardware and posting to queues adds to the total cycle time, causing the sampling period to drift beyond 2000 ms. In contrast, `vTaskDelayUntil()` tracks an absolute reference timestamp (`lastWakeTime`). It dynamically adjusts the sleep duration based on task execution time, ensuring strict periodic execution without timing drift—a requirement for reliable sampling.

## 5. Verification and Testing

### 5.1 Automated Unit Testing Results (13 Tests Passing)

| Category     | Test Case Description           | Input Parameters               | Expected Result      | Status |
|--------------|---------------------------------|--------------------------------|----------------------|--------|
| Alarm Logic  | Below Lower Threshold           | 17.9 °C                        | LOW_TEMPERAT<br>URE  | PASS   |
| Alarm Logic  | Exactly Lower Boundary          | 18.0 °C                        | NORMAL               | PASS   |
| Alarm Logic  | Normal Operating Range          | 24.5 °C                        | NORMAL               | PASS   |
| Alarm Logic  | Exactly Upper Boundary          | 30.0 °C                        | NORMAL               | PASS   |
| Alarm Logic  | Above Upper Threshold           | 30.1 °C                        | HIGH_TEMPERAT<br>URE | PASS   |
| Navigation   | Forward Transition 1            | TEMP + CW turn                 | HUMIDITY             | PASS   |
| Navigation   | Forward Transition 2            | HUMIDITY + CW turn             | LIGHT                | PASS   |
| Navigation   | Forward Wraparound              | MOTION + CW turn               | TEMP                 | PASS   |
| Navigation   | Reverse Wraparound              | TEMP + CCW turn                | MOTION               | PASS   |
| System State | Active State Without Timeout    | ACTIVE, 5s elapsed             | ACTIVE               | PASS   |
| System State | Active State Inactivity Timeout | ACTIVE, 15s elapsed, no motion | INACTIVE             | PASS   |
| System State | Inactive Idle State             | INACTIVE, no motion            | INACTIVE             | PASS   |
| System State | Inactive Reactivation           | INACTIVE, PIR signal high      | ACTIVE               | PASS   |

### 5.2 Wokwi Functional Verification Record

| Test ID | Stimulus / Input                | Expected Behavior            | Actual Observed Behavior                       | Result |
|---------|---------------------------------|------------------------------|------------------------------------------------|--------|
| FT-01   | Adjust DHT22 slider to 28.0 °C  | Display updates temperature  | OLED rendered 28.0 °C within 2s                | PASS   |
| FT-02   | Adjust DHT22 slider to 65% RH   | Display updates humidity     | OLED updated humidity to 65.0%                 | PASS   |
| FT-03   | Adjust LDR potentiometer        | Relative light level updates | Light reading scaled properly (0–100%)         | PASS   |
| FT-04   | Rotate encoder knob clockwise   | Display cycles forward       | Display cycled: Temp -> Hum -> Light -> Motion | PASS   |
| FT-05   | Rotate encoder counterclockwise | Display cycles backward      | Page cycled in reverse with proper wraparound  | PASS   |

|       |                                   |                              |                                                      |      |
|-------|-----------------------------------|------------------------------|------------------------------------------------------|------|
| FT-06 | Increase DHT22 temp > 30.0 °C     | Alarm buzzer sounds          | Buzzer activated immediately upon queue receive      | PASS |
| FT-07 | Lower DHT22 temp back to 24.0 °C  | Alarm buzzer silences        | Buzzer de-asserted instantly                         | PASS |
| FT-08 | Trigger PIR motion sensor         | System enters/remains ACTIVE | Display illuminated; system maintained active status | PASS |
| FT-09 | Leave system idle for > 15s       | System enters INACTIVE mode  | Display blanked after exactly 15 seconds             | PASS |
| FT-10 | Trigger PIR motion while INACTIVE | System reactivates to ACTIVE | OLED restored screen display immediately             | PASS |

### 5.3 Deliberate FreeRTOS Fault Experiments

- **Fault Experiment 1 (Remove Blocking Delay):** Removed `vTaskDelayUntil()` from `SensorTask` while keeping its priority at 2. Observation: CPU starvation occurred for lower priority tasks (`DisplayTask`, priority 1). OLED screen updates completely froze because `SensorTask` monopolized core execution in an unblocked loop.
- **Fault Experiment 2 (Elevated Task Priority):** Assigned Priority 4 (highest) to `DisplayTask` during continuous re-renders. Observation: Display driver calculations delayed queue processing in `InputTask`, causing rotary encoder navigation to feel sluggish.
- **Fault Experiment 3 (Remove Serial Mutex Protection):** Removed `xSerialMutex` around concurrent logging functions. Observation: Serial Monitor logs became corrupted with interleaved and fragmented text strings during simultaneous output operations.

## 6. Static Code Analysis (pio check)

| Finding                                   | File & Line    | Severity | Root Cause                                                          | Resolution                                                                          |
|-------------------------------------------|----------------|----------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Unused variable 'raw_adc_val'             | sensors.cpp:42 | Warning  | Leftover variable from earlier uncalibrated ADC tests.              | Removed redundant variable declaration.                                             |
| Implicit conversion float to uint8_t      | alarm.cpp:18   | Warning  | Passing floating point temp directly to integer threshold function. | Added explicit cast <code>static_cast&lt;uint8_t&gt;()</code> after bounding check. |
| Variable 'xLastWakeTime' set but not used | motion.cpp:25  | Style    | Debug variable retained after refactoring.                          | Cleaned up unused local variable.                                                   |

## 7. Architectural Trade-offs and Discussion

A primary technical hurdle involved maintaining responsive user interactions alongside high-latency I2C updates to the SSD1306 OLED screen. To resolve this latency issue, display refreshing was isolated within a dedicated DisplayTask driven via inter-task queue events. Consequently, prolonged rendering routines were prevented from stalling time-critical telemetry acquisition routines or encoder input handling.

Furthermore, core evaluation routines—including threshold verification and operating state machine logic—were decoupled from low-level peripheral drivers. This design modularity enabled comprehensive unit testing of state transitions and safety limit evaluation locally on a host computer without relying on hardware or Wokwi emulation.

## 8. Requirements Traceability Matrix

| Requirement ID | Implementation Subsystem       | Verification Evidence                  |
|----------------|--------------------------------|----------------------------------------|
| FR-01          | SensorTask (sensors.cpp)       | FT-01                                  |
| FR-02          | SensorTask (sensors.cpp)       | FT-02                                  |
| FR-03          | SensorTask (sensors.cpp)       | FT-03                                  |
| FR-04          | MotionTask (motion.cpp)        | FT-08, FT-10                           |
| FR-05          | DisplayTask (display.cpp)      | FT-01, FT-02, FT-03                    |
| FR-06          | InputTask (input.cpp)          | Navigation Unit Tests + FT-04, FT-05   |
| FR-07          | AlarmTask (alarm.cpp)          | Alarm Unit Tests + FT-06, FT-07        |
| FR-08          | SystemState (system_state.cpp) | System State Unit Tests + FT-08, FT-09 |
| FR-09          | MotionTask / SystemState       | System State Unit Tests + FT-09        |
| FR-10          | MotionTask / SystemState       | System State Unit Tests + FT-10        |

## 9. Conclusion

This laboratory successfully implemented a FreeRTOS-based multisensor room monitoring system on an ESP32 using PlatformIO and ESP-IDF. Preemptive multitasking, mutex resource locking, queue-based message passing, and deterministic state management enabled stable real-time operation. Future improvements include implementing low-power light-sleep modes during the INACTIVE state and integrating hardware-calibrated lux sensors.